Select, Don't Construct: Verbatim KV-Memory for Long-Horizon Agents

Zefeng Cai
3294005475a@gmail.com

Abstract

Long-horizon agents—coding, web, embodied, multi-session chat—accumulate interaction histories that overflow any context window. Many deployed memory systems answer this by constructing a lossy store—extracting atomic facts, building knowledge graphs, or summarizing—and retrieving from it; on dense, objective agent trajectories this destroys exactly the evidence a question needs. kvmemory takes the opposite route: keep the raw turns, route each query to the spans it needs, and rehydrate them verbatim—a small, readable framework (recency-tiered verbatim store, pluggable router, verbatim rehydration) for self-hosted inference.

Our central finding is that payload fidelity—not memory construction, and not router sophistication—is the dominant variable. On AMA-Bench (official Qwen3-32B harness, full 2,496 QA), under the harness's default answer prompt, the memory stack alone reaches 0.5954: above the purpose-built AMA-Agent (+2.3 pp), with every prior memory system +10–35 pp behind; a disclosed, separately ablated structured answer instruction brings the end-to-end system to 0.6466 in our run (leaderboard submission pending). A same-batch memory×reader factorial and matched payload ablations locate the gains: lossily re-encoding the same selected evidence costs 5–14 pp; router sophistication is second-order (+1.4 pp semantic, +1.7 pp structural) and regime-dependent (+12.6 pp on diffuse dialogue); and the largest single knob is the reader instruction (+5.1–5.8 pp at either router)—so memory comparisons that do not control the reader largely measure reader differences. The finding replicates on LOCOMO and LongMemEval-S within a stated regime boundary.

Because the store is verbatim KV, selection is also a serving primitive: prefill the trajectory once, gather the selected spans' cached KV at their original positions, and re-prefill only the question—faithful to a full-prefill-then-mask oracle in our tests, and 1.6–10.9× faster to first token as the selected evidence grows (a prototype microbenchmark; decode still attends over the selected KV).

The recipe wins when history overflows the window or distractors drown the reader; full context can win when it fits, and construction-time extraction can win for a weak reader on tiny isolated facts. We map these boundaries and release configurations, predictions, and judge outputs as a reproducible baseline that centers payload fidelity over memory construction.

1 Introduction

A long-horizon agent's state is its history: the tool calls it issued, the observations it received, the reasoning it wrote. As that history grows past the model's context window, the agent needs a memory. The prevailing answer is to construct one—to run an LLM over the trajectory that extracts atomic facts (Mem0 (Chhikara et al., 2025)), writes and links notes (A-Mem (Xu et al., 2025)), distills hierarchical summaries (MemoryBank (Zhong et al., 2024)), or builds a temporal knowledge graph (Zep (Rasmussen et al., 2025))—and then retrieve from that constructed, lossy store. This is natural for dialogue memory, where the

salient content is a handful of stable user facts. It is a poor fit for agent trajectories, where the answer is often an exact value, a step index, a code diff, or an SQL filter that a fact-extractor paraphrases away. AMA-Bench (AMA-Bench Team, 2026) makes this concrete: purpose-built dialogue-memory systems (Mem0 0.21, A-Mem 0.32, MemGPT 0.33, MemoryBank 0.34) all score below a no-memory long-context baseline (0.52) on dense agent trajectories—lossy construction is the bottleneck.

Our claim is a positioning one: for these workloads, a deliberately simple alternative to construction is surprisingly strong. A KV cache built over the full trajectory is already a usable associative memory of it; the operation we want is not to compress it but to select which raw spans a query needs and to rehydrate them verbatim. Algorithmically this is close to query-aware raw-chunk retrieval; the contribution is the empirical case that raw-span memory beats constructed memory on agent trajectories, plus the systems mechanism that makes it cheap. Selection denoises (it drops the irrelevant turns that drown the answer) while preserving the exact tokens that construction would lose, and it is cheap to deploy on self-hosted inference (vLLM (Kwon et al., 2023), SGLang (Zheng et al., 2024)) because the query-independent part of the context is prefilled once and reused. The claim is deliberately conditional, and we keep it so throughout: select verbatim when the history overflows the window or distractors drown the reader; full context can win when it fits and the reader can exploit it; construction-time extraction wins only for a weak reader over tiny isolated facts (§6).

We build this into kvmemory (§2) and evaluate it on three benchmark families (§4). Our contributions:

1. A finding with design consequences: payload fidelity dominates (§4.1, §4.4). Selecting raw spans and re-injecting them verbatim beats every published lossy-construction system by +10–35 pp cross-system, and matched-evidence ablations make the payload's role causal: re-encoding the same selected evidence as summaries or facts costs 8–14 pp in a stripped oracle setting and −5.1 pp inside the deployed pipeline.
2. A memory×reader confound audit (§4.4): a same-batch factorial separates memory-side from reader-side gains—routing refinements are second-order (+1.4 pp semantic / +1.7 pp structural on AMA; regime-dependent: +4.3 pp on LongMemEval, +12.6 pp on LOCOMO) while a one-sentence reader instruction is worth +5.1–5.8 pp at either router—plus an eight-mechanism reader-side stress test showing the memory side saturates once routing reaches $\sim 100\%$ recall. Memory comparisons that do not control the reader largely measure reader differences.
3. Results on official harnesses (§4.1, §4.2, §4.3): 0.5954 under the AMA-Bench default reader and 0.6466 end-to-end on our run of the official harness over the full 2,496 QA (above the prior leaderboard best 0.6246; submission pending); competitive LOCOMO (0.704 vs Mem0 0.671 / Zep 0.660 under the public protocol, positioned as an out-of-domain check); LongMemEval-S within a stated regime boundary.
4. A serving mechanism: literal KV sub-selection (§4.5): selected spans are served by gathering their cached KV at original positions, not re-encoded. It reproduces a full-prefill-then-mask oracle (100% first-token argmax on synthetic and real AMA trajectories, and on a non-Qwen backbone), is accuracy-neutral vs the text pipeline on real AMA (tied within noise under a consistent reader/judge), and runs 1.6–10.9× faster to first token as selected evidence grows (prototype microbenchmark), on top of ∼2M prefill tokens saved per full AMA run.
5. A regime map and a forkable baseline (§2, §6): a small, readable framework, plus the boundaries—verbatim selection wins with a capable reader over dense evidence; full context can win when it fits; construction-time extraction wins only for a weak reader over tiny isolated facts.

2 Method: kvmemory

kvmemory is a KV-cache memory manager for self-hosted long-horizon agents. The design goal is a readable base others can fork: keep raw context as reusable KV and select per

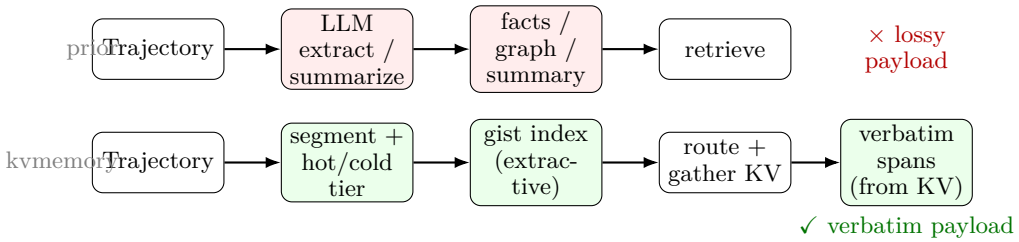

Figure 1: Prior memory systems construct a lossy store (extract/summarize/ graph) and retrieve from it. kvmemory keeps raw segments, builds only a cheap extractive gist index over the cold ones, and at query time routes to and gathers the selected spans' KV verbatim—gist routes, verbatim answers.

query, instead of lossily compressing (Figure 1). Algorithm 1 gives the build-once / retrieve-per-query loop and Table 1 its default configuration; every row of that table is a swap point, and the paragraphs below walk the components in turn.

Segments and three recency tiers. The trajectory is flattened into Segments (for agent traces, one per turn: action / observation; for dialogue, one per utterance/session with its timestamp). At query time the segments occupy three tiers. Hot: the most recent segments, kept verbatim (a recency prior). Warm: older segments rehydrated verbatim when the router selects them for the current query. Cold: every other old segment, represented by a lightweight gist—a cheap extractive snippet (the head of the action/observation), not an LLM summary. The gist plays two roles: it is the routing index, and in the assembled context each un-selected cold turn appears only as its one-line gist inside a recency overview—navigation scaffolding, so the reader can see the shape of the whole trajectory. The answer-bearing content is served verbatim: recent turns in full, plus the router-selected turns rehydrated in full as an appendix. Any turn the query actually needs is thus upgraded to verbatim; a gist is never the payload for selected evidence. Even the index is non-generative: we construct for indexing and scaffolding but answer from verbatim. The slogan is gist routes, verbatim answers. Both halves of this layout earn their tokens: cutting the overview collapses narrative-heavy domains (−27 pp on Embodied in the budget sweep of §4.4), while answering from lossy re-encodings instead of verbatim spans costs 8–14 pp (Fig. 3).

The router (pluggable; its payoff is regime-dependent). A Router.select(query, candidates, k) interface returns which old segments to rehydrate. In order of cost: Lexical (token overlap, BM25-like; free, zero-dependency, already strong); Embedding (dense retrieval with a lexical pre-filter to bound cost); Hybrid (reciprocal-rank fusion of the two, which tops both components); and a step-pinning variant that additionally pins the steps a question names ("step N", turn references)—a cheap structural cue, not a fancier scorer. We view step-pinning as a minimal instance of structural address resolution: agent trajectories are not unstructured documents but event logs with stable addresses (step and turn indices, tool-call ids, file paths, URLs), and when a query names an address the memory should dereference it deterministically—reserving content-based routing for the remaining budget—rather than force all access through semantic similarity. It is a second access path alongside content addressing, not a benchmark-specific heuristic; we report its contribution separately throughout (§4.4). How much router sophistication helps is regime-dependent (§4.4); on step-indexed agent trajectories a cheap router already saturates, and the verbatim substrate does the work.

Verbatim rehydration is the substrate. Selected segments are rehydrated byte-for-byte, not summarized. Lossy construction destroys dense objective information (exact values, code, IDs); verbatim selection avoids that and, by dropping irrelevant turns, also denoises relative to dumping the full context (lost-in-the-middle). Under a hard budget, a query-focused per-step trim keeps only the query-relevant lines of a giant observation—a rule/lexical line filter that preserves the original lines verbatim (no paraphrase), so "verbatim" means the

retained lines are exact, not that the whole segment is always kept—letting more of a multi-hop chain fit. This trim fires only when a single observation overflows the budget; because it removes surrounding lines, it can drop context a question needs (e.g. in a stack trace, diff, or DOM)—a failure mode we flag rather than hide, and "verbatim" should be read as retained evidence is verbatim, not the full observation is always retained.

KV reuse by sub-selection (the efficiency claim). Because the store is verbatim KV, selection is a cache operation, not a prompt operation. We prefill the episode's trajectory once into a key–value cache, recording each segment's token span. At query time the router names the segments to keep; we gather their cached KV at the original positions (position-preserving, so RoPE encodes the true inter-span distances—renumbering would require re-rotating the stored keys) and decode the question on top, re-prefilling nothing but the question itself. The selected verbatim spans are thus served from the cache, never re-encoded. Because RoPE is relative, keeping spans at their original positions is exact for the retained-and-earlier tokens: this sub-selected prefill agrees with a full-prefill-then-mask-dropped oracle on the first decoded token in all our checks (§4.5). The per-query prefill is therefore $O(\text{question})$ rather than $O(\text{selected evidence})$—the selected spans are not re-encoded, though the decode still attends over their KV—so the more a multi-hop query pulls, the larger the prefill saving. Self-hosting gives full control of this reuse; a prefix-friendly layout additionally benefits from the prompt-caching that production APIs now expose.

Assembly: two modes. The deployed benchmark (text) pipeline assembles $[\,\text{recency overview} \,\|\, \text{selected warm turns, verbatim} \,\|\, \text{question}\,]$, where the overview = hot turns verbatim + every cold turn as its one-line gist. The overview is query-independent (identical for every query in an episode, so its KV is prefilled once and reused); the evidence appendix and question vary per query. All headline benchmark numbers use this layout; the budget sweep and gist-only ablations (§4.4) probe its two halves. The compact KV-serving mode evaluated in §4.5 drops the overview and serves $[\,\text{selected turns in temporal order} \,\|\, \text{question}\,]$ (hot + router-selected warm, all verbatim, no gist), gathering the selected spans' cached KV at their original positions. In both modes every turn keeps its explicit <step N> tag, so the reader reconstructs the timeline from tags rather than physical position: selecting a subset in temporal order does not scramble temporal reasoning, and the recent (hot) turns are never displaced by older selected ones. The cacheable object is not a fixed text prefix—the selected set varies per query—but the one-time trajectory prefill, from which each query gathers its spans' KV at their original positions (§4.5); a hosted API additionally reuses the stable $[\,\text{system} \,\|\, \text{task}\,]$ header via contiguous-prefix caching.

Answer prompt. The reader answers over the assembled context with a fixed prompt; we evaluate both the harness default and a one-sentence structured instruction, and report them as a separately ablated reader-side variable throughout (§4.4).

What it is not. Not a lossy fact/graph/summary memory (we keep raw spans); not a KV eviction/compression scheme (we select and rehydrate at full precision); its arbitrary-span KV reuse is realized self-hosting, though the prefix-friendly layout also benefits hosted prompt-caching.

3 Experimental setup

We evaluate on three long-term-memory families that stress different inputs: AMA-Bench (AMA-Bench Team, 2026) (agent execution trajectories), LOCOMO (Maharana et al., 2024) (multi-session dialogue), and LongMemEval-S (Wu et al., 2025) (long-context chat-assistant histories). Unless noted, the reader and the LLM-as-judge are the same model as each benchmark's official protocol, and we report each benchmark's own metric.

For AMA-Bench we run the official run.py harness end to end (memory construction → per-query retrieval → answer → LLM-as-judge) on the full open-ended set (208 episodes /

Algorithm 1 kvmemory: build once, retrieve (and gather KV) per query.

procedure BuildMemory(trajectory T)
  segment T into turns; store each raw payload s_i (action/observation) verbatim
  for each s_i: $\text{gist}_i \leftarrow \text{ExtractHead}(s_i)$ ▷ cheap extractive index, not an LLM summary
  prefill the flattened T once into KV cache C; record each s_i's token span
  return $(\{s_i\}, \{\text{gist}_i\}, C, \text{spans})$
end procedure
procedure Retrieve(query q, budget B)
  $H \leftarrow$ the h most recent turns (hot tier, kept verbatim)
  $P \leftarrow$ turns named by structural refs in q ("step N", "turns N–M")
  score the remaining (cold) turns by router $R(q, \cdot)$ ▷ lexical / embedding / RRF hybrid
  $K \leftarrow$ top-k of $P \cup$ scored under budget B; line-trim oversized turns if needed
  gather C's KV for $H \cup K$ at their original positions (position-preserving)
  decode q over $[\, H \,\|\, K \,]$ verbatim (un-selected cold turns dropped), re-prefilling only q
end procedure

Table 1: kvmemory default configuration (the fork surface; every row is swappable).

Component	Default setting
Hot tier	h=4 most recent turns, verbatim
Cold gist	extractive head: action[:120] + obs[:160] chars (no LLM)
Lexical router	token overlap over [a-z0-9]+ tokens (BM25-like), zero-dependency
Embedding router	Qwen3-Embedding-0.6B, last-token pool + L2, cosine top-k, lexical pre-filter
Hybrid router	RRF of {lexical, embedding}, c=60, $4k$ candidates per sub-router
Step-pin	regex (turn|step)s? N(–M)?; pin named turns verbatim
Rehydration k	top-5 selected turns (budget-bounded)
Context budget	22k tokens (32k window − 10k thinking reserve); left-truncate old first
Answer prompt	structured: sub-question → cite exact evidence → compose
Reader / judge	Qwen3-32B (official AMA-Bench Track B)

2,496 QA, six domains), with backbone and judge both Qwen3-32B (Qwen Team, 2025) (the official "Track B"), so our numbers drop directly into the published table; context is capped at 22k tokens (the 32k window minus the thinking reserve). For LOCOMO we report both the public gpt-4o-mini leaderboard protocol (comparable to Mem0/Zep) and a consistent-judge regime (reader = judge = Qwen3-32B) that removes the cross-paper judge confound. For LongMemEval-S we report only the Qwen3-32B target-reader regime (§6 explains the scoping).

4 Results

4.1 AMA-Bench: leading results on agent-trajectory memory

Table 2 places kvmemory on the AMA-Bench leaderboard. With only a free lexical router, kvmemory already matches the purpose-built AMA-Agent (0.5621 vs 0.5722) and beats every other published memory system by +10 to +35 pp—confirming the core thesis that verbatim selection over raw turns is a stronger substrate than lossy construction or similarity retrieval. Memory-side improvements alone—the hybrid router plus the structural step-pin (§4.4), still under the harness's default answer prompt—reach 0.5954, +2.3 pp above AMA-Agent (95% bootstrap CI $[0.576, 0.615]$ over QA; $[0.571, 0.620]$ under the more conservative episode-cluster resampling, whose lower bound sits marginally below AMA-Agent's 0.5722—we therefore claim "above", not "significantly above"). Our headline run additionally swaps the reader's one-sentence answer instruction for a structured one ("break the request into sub-questions, answer each citing exact evidence verbatim, then give the final answer"), which we disclose and ablate as a reader-side variable, orthogonal to the memory (§4.4):

Table 2: AMA-Bench, average accuracy over the full 2,496 QA (Qwen3-32B backbone and judge, official harness). Baseline rows are the published AMA-Bench numbers; kvmemory rows are our runs on the same harness.

Method	Avg. accuracy
kvmemory (step-pinning router)	0.6466 (our run; pending official verification)
HybridFocus (prior leaderboard best, unreleased)	0.6246
AMA-Agent (purpose-built)	0.5722
kvmemory (lexical router, zero-dependency)	0.5621
MemoRAG (Qian et al., 2024)	0.4606
HippoRAG2 (Gutiérrez et al., 2024)	0.4480
BM25	0.3436
MemoryBank (Zhong et al., 2024)	0.3397
MemGPT (Packer et al., 2023)	0.3304
A-Mem (Xu et al., 2025)	0.3186
Mem0 (Chhikara et al., 2025)	0.2104
Long-context Qwen3-32B (no memory)	0.52

Table 3: AMA-Bench per-domain accuracy: kvmemory (step-pinning) vs the purpose-built AMA-Agent. kvmemory wins five of six domains.

	TEXT2SQL	SOFTWARE	WEB	GAME	EMBODIED	OPENWORLD
kvmemory (pin)	0.740	0.475	0.694	0.683	0.575	0.681
AMA-Agent	0.609	0.636	0.517	0.644	0.499	0.471

with it, kvmemory reaches 0.6466 on our run of the official harness, above AMA-Agent (+7.4 pp) and the prior (unreleased) leaderboard best, HybridFocus 0.6246 (leaderboard submission pending; we release configs, raw predictions, and judge outputs). AMA-Bench ships only a test split, so we disclose that all tuning necessarily happened on it—for every system on the leaderboard alike; our defense against overfitting is consistency across three benchmark families and released artifacts, not a held-out split that does not exist. Per domain (Table 3) kvmemory beats AMA-Agent in five of six domains; the exception is Software, which we dissect below.

A faithful re-run of the full harness reproduces this headline (0.643 under the Qwen3-32B judge; 95% bootstrap CI $[0.623, 0.662]$ over the 2,496 QA), and addresses the natural worry that a Qwen reader graded by a Qwen judge inflates the score. Re-judging that run's identical predictions with an independent Llama-3.1-70B judge—a different model family—yields 0.677 $[0.658, 0.695]$ with 84% item-level agreement: on our predictions, the Qwen judge is the stricter of the two. We note the official AMA-Bench documentation reports a leniency bias for Qwen3-32B as a judge in absolute terms; since every leaderboard system is scored by the same judge, our comparisons are on equal footing, and we treat the cross-family re-judge as supportive rather than definitive. The per-type gap is concentrated where the reader, not the router, is the bottleneck: on Software the accuracy drop is uniform across question types (0.37–0.61), and two probes pin the cause down. First, recall: on all 268 step-referencing Software questions the asked step is present in the selected context (100%), and every error is a reader-side failure. Second, access: giving the reader deterministic lookup tools over the lossless store (exact get-by-index, exhaustive search) leaves accuracy unchanged (+0.7 pp, same-batch; §4.4). Third, format: forcing the reader to answer by copying exact strings verbatim (a quote-first instruction) makes things far worse (−10.0 pp, 0.363 vs 0.463, same batch)—suppressing the reader's own reasoning costs much more than paraphrase penalties ever did. So the gap is not retrieval, not access, and not answer formatting. What remains is AMA-Agent's iterative answer-production loop—retrieve, execute code over the full trajectory, verify, compose from tool outputs: a process, not a format. The reasoning-heavy residue is the same failure mode we quantify on LongMemEval (§4.3), where 91% of the router's misses are reasoning failures with the gold evidence already in context. We treat this as a complementary paradigm, not a deficiency the memory can absorb (§6).

Table 4: LOCOMO. Left: public gpt-4o-mini protocol (comparable to Mem0/Zep). Right: consistent Qwen3-32B judge. "Full-history" prompts the entire dialogue (distractors included); it is a strong baseline, not a gold-evidence oracle.

Public (gpt-4o-mini)		Consistent judge (Qwen3-32B)	
System	J	Arm	J
Full-history prompt	0.756	kvmemory (hybrid retrieval)	0.716
kvmemory (hybrid retrieval)	0.704	Full-history prompt	0.661
Mem0 (Chhikara et al., 2025)	0.671	kvmemory (lexical retrieval)	0.590
Zep (Rasmussen et al., 2025)	0.660		

4.2 LOCOMO: competitive with dedicated dialogue-memory systems

On LOCOMO's public protocol (reader = judge = gpt-4o-mini, the Mem0/Zep convention), kvmemory's compact hybrid-retrieval arm reaches $J = 0.704$, beating Mem0 (0.671) and Zep (0.660)—though under this same gpt-4o-mini judge full-history prompting scores higher still (0.756), so we do not claim to beat full context here. The two judges disagree on exactly that comparison, which is why we re-run it under a single consistent reader/judge (Qwen3-32B, which removes the cross-paper judge confound): there, compact selection does edge out full-history prompting ($0.716 > 0.661$), because dropping the irrelevant turns denoises—note a full-history prompt is not an oracle, since it carries the distractors that hurt the reader. Verbatim memory also abstains correctly on the adversarial, unanswerable category 87–89% of the time (vs 75% for full context), because it knows whether evidence exists.

We position LOCOMO as an out-of-domain sanity check, not a headline claim: the benchmark has known protocol variants (QA subsets, judge choices), and newer file- and tool-based systems report stronger numbers under their own setups (e.g. a filesystem-backed agent at 0.740 (Letta Team, 2025) and 0.744 (Pam Research, 2025)). The useful signal for our thesis is narrower and survives those caveats: under one fixed protocol, compact verbatim selection is competitive with dedicated dialogue-memory systems, and under a consistent judge it beats prompting the full history—on dialogue, the workload construction was designed for.

4.3 LongMemEval-S: generalization with a clear boundary

LongMemEval-S puts each question over its own ∼115k-token, ∼50-session haystack; we evaluate the full 500 questions (470 answerable + 30 abstention), with 95% bootstrap CIs, on the official longmemeval_s release (xiaowu0162/longmemeval; the benchmark's oracle evidence-session labels are used only for error attribution between selection and reader failures, never by the method; exact file and access date ship with our artifacts). In the Qwen3-32B regime two facts hold. (i) When the history exceeds the window—the operative case for long-horizon agents—the full arm must truncate, and routed selection at a fraction of the budget beats it decisively: $0.496\,[0.453, 0.543]$ vs $0.302\,[0.262, 0.343]$ at a 28k budget ($+0.194\,[0.149, 0.238]$). The mechanism is clean: truncation drops old evidence sessions (95% of its answerable misses are selection failures), whereas the router keeps the gold session in budget (9% selection-fail). (ii) When the history instead fits an extended 128k window (via YaRN), full context wins overall ($0.564\,[0.519, 0.609]$ vs 0.496; $+0.068\,[0.019, 0.119]$)—but at 4.5× the context, and its edge is confined to recall/breadth: full leads on single-session recall (ss-user 0.828 vs 0.609, ss-asst 0.946 vs 0.768) and multi-session (0.405 vs 0.322), while routed selection wins temporal-reasoning (0.331 vs 0.291) and knowledge-update (0.736 vs 0.708; Fig. 2). So "selection ≥ full" is regime-dependent, and we state the crossover rather than cherry-pick a side. Here the router is load-bearing (unlike on AMA): the dense-embedding hybrid beats a lexical router (0.496 vs 0.453; $+0.043\,[0.006, 0.079]$), entirely by cutting selection failures (9% vs 23%).

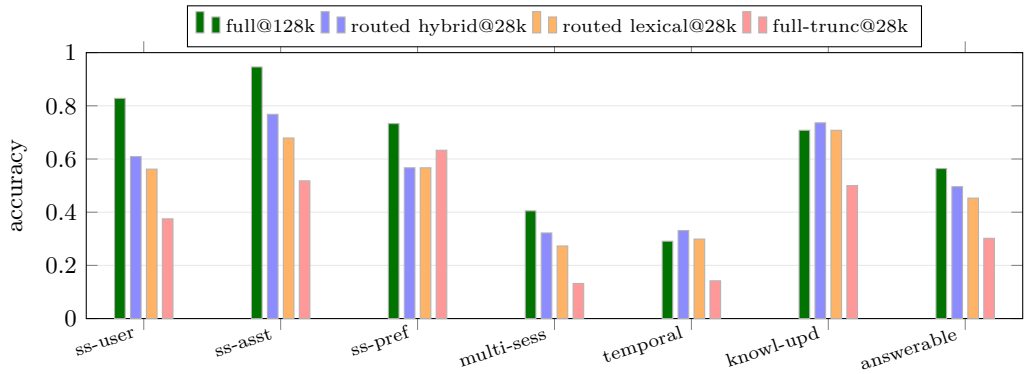

Figure 2: LongMemEval-S per-type accuracy, full 500 questions (Qwen3-32B served at 128k via YaRN; internal Qwen3-32B judge; 95% bootstrap CIs on answerable in §4.3). Two regimes: when the 115k haystack fits the window, full context wins overall (answerable 0.564), but routed selection at 4.5× less budget leads on temporal and knowledge-update; at a 28k budget where the history must truncate, routed selection (0.496) beats truncation (0.302) by +19 pp, and the embedding-hybrid router beats lexical by +4 pp. Correct-abstention (not shown) is 0.60/0.67/0.67/0.77.

Table 5: Same-batch memory×reader factorial on AMA-Bench (full 2,496 QA, official harness; 95% QA-level bootstrap CIs; episode-cluster CIs—208 clusters—are ∼30% wider, e.g. [0.571, 0.620] for the default-reader memory stack). The two axes are additive to within batch noise: routing adds +3.1–3.5 pp at either reader, and the one-sentence structured answer instruction adds +5.1–5.8 pp at either router. Under the default reader the memory stack alone already scores above the purpose-built AMA-Agent (0.5722).

Memory configuration	Default answer prompt	Structured answer prompt
Lexical router	0.5601 [0.541, 0.579]	0.6154 [0.595, 0.635]
+ hybrid router + step-pin	0.5954 [0.576, 0.615]	0.6466 [0.623, 0.662]

4.4 Where the gain comes from: substrate, not router

It would be tempting to credit a clever router. A same-batch memory×reader factorial (Table 5) says otherwise. The verbatim-selection substrate is the hero—just selecting raw spans, with a token-overlap router and the harness's default reader, already matches the purpose-built system (0.5601 vs 0.5722), and the full memory stack under that same default reader beats it (0.5954, +2.3 pp). Routing refinements are second-order: the dense-embedding hybrid adds +1.4 pp and the structural step-pin +1.7 pp, consistently at either reader (+3.1–3.5 pp total). The single largest effect is not in the memory at all: swapping the reader's one-sentence answer instruction for a structured one adds +5.1–5.8 pp at either router, and the two axes are additive to within batch noise (same-day replicate ±0.5 pp). Memory systems ship with very different readers (AMA-Agent an agentic tool loop, MemGPT its own control loop), so end-to-end gaps that do not control the reader conflate the two effects; we report them separately. Router sophistication remains regime-dependent (Table 6): its payoff scales with how lexically un-addressable the evidence is—modest on step-indexed agent trajectories, load-bearing on diffuse dialogue (where, on LOCOMO, it is the only thing that lifts selection above full context). This is a sharper and more honest statement than "the router is the lever."

Substrate, then structure, then semantics. On AMA-Bench specifically, three memory-side effects separate cleanly (Table 7). The substrate—selecting raw spans at all, even with a free token-overlap router—is the bulk of the gain, lifting a lossy-construction field into AMA-Agent territory. Semantic router sophistication (lexical→dense-hybrid) adds a second-order +1.4 pp. The remaining lever is not a smarter scorer but a one-line structural cue: 73% of AMA-Bench questions cite a step index ("at Turn 38", "across steps 28–30"), and pinning exactly those steps adds +1.7 pp—and does so entirely on the step-citing questions (+2.5 pp on them vs exactly +0.0 pp on the rest), a clean mechanism signature. We disclose this

Table 6: Router sophistication (lexical → dense-embedding hybrid) is regime-dependent: its payoff grows with how lexically un-addressable the evidence is.

Benchmark (evidence style)	Δ from the hybrid router
AMA-Bench (step-indexed trajectories)	+1.4 pp
LongMemEval-S (long chat)	+4.3 pp
LOCOMO (diffuse dialogue)	+12.6 pp

Table 7: AMA-Bench ablation deltas (full 2,496 QA, same-batch). The verbatim substrate carries the result; semantic and structural routing refinements are second-order; the reader-side answer instruction—listed for scale, and reported separately because it is not a memory effect—is larger than both combined.

Component	Δ Avg. accuracy
Substrate: raw spans vs. lossy construction (cross-system)	+10 to +35 pp
controlled (same oracle evidence; Fig. 3)	+8 to +10 pp
Semantic router (lexical → dense-embedding hybrid)	+1.4 pp
Structural step-pin (+2.5 pp on the 73% cited-step questions, +0.0 else)	+1.7 pp
Reader-side, for scale: structured answer instruction	+5.1–5.8 pp

benchmark property rather than lean on it silently: on step-indexed agent traces the order is substrate ≫ semantic router ≈ structure, and the structural pin is a transparent rule, not a learned exploit.

Is the result a step-citing artifact? (robustness). Largely not, and we cross-check with a second judge. Controlling on the same Qwen3-32B reader and judge over 910 step-citing questions, routing the question with its step number deleted costs lexical selection only −0.2 pp (28.6 → 28.4)—so selection locates the evidence by content, not by the index—and a different-family judge (Llama-3.1-8B) re-scoring the same answers agrees (−1.0 pp), so this is not a judge artifact. The cited step is informative when present: pinning it alone reaches 38.5% vs 28.6% for lexical top-5 (and 40.1 vs 36.0 under the Llama judge—ordering preserved), because it is the correct evidence, not a numeric shortcut. Corrupting the resolved address completes the picture: shifting every pin three steps off its target leaves accuracy unchanged (0.630 vs 0.629, same batch)—the near-miss keeps the referenced neighborhood in context and content routing independently recovers the exact step—while replacing every pin with a far-random wrong step costs −1.8 pp (0.611), roughly the pin's own contribution forgone, with content routing holding the floor. Structural address resolution is thus a cheap redundant guarantee that degrades gracefully under corruption, not a fragile dependency. We are careful about one boundary: non-step questions remain answerable but are genuinely harder, and by how much is judge-dependent (within −2.4 pp of step-citing under the Qwen judge, but −14.8 pp under Llama), so we claim robustness to the step cue—not that non-step questions are equally easy. The step-pin is thus a transparent bonus on the step-citing subset, not the source of the result.

Controlled payload ablation (oracle evidence). The cross-system +10–35 pp above conflates payload form with chunking, prompt, and budget. We isolate the payload with a within-system test: on step-citing questions we take the exact cited step as oracle evidence (holding retrieval fixed) and present that same evidence three ways—the raw verbatim step, an LLM summary, or LLM-extracted facts—to the same Qwen3-32B reader, prompt, and judge. Only the payload form varies, so the gap is causal (Fig. 3): verbatim beats an LLM summary by +10.1 pp and extracted facts by +8.5 pp over 910 QA, widening to +14 pp on exact-recall (type A) where lossy forms drop the exact value. This is the controlled version of "the substrate is the hero": matched evidence, matched reader, lossy re-encoding alone costs 8–14 pp. To rule out the reader grading its own generations leniently, a different-family judge (Llama-3.1-8B) re-scoring the same answers preserves the ordering—verbatim 34.3% > facts 30.2% > summary 29.8%, still +4–5 pp—so the verbatim advantage is not a judge artifact. (These absolute numbers are deliberately not comparable to Table 2: in this

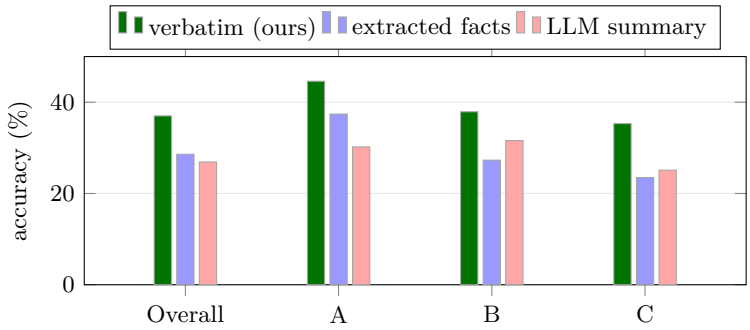

Figure 3: Controlled payload ablation (matched oracle evidence, reader, prompt, and judge; 910 step-citing AMA-Bench QA; type A = exact-recall)—only the payload form varies. Re-injecting the selected steps verbatim beats lossy re-encodings on every question type: +10.1/+8.5 pp overall over LLM-summary/extracted-facts, widening to +14 pp on exact-recall. (On the degenerate type D, omitted, all three sit at $\sim 2\%$.) Lossy re-encoding of matched evidence alone costs 8–14 pp—isolating the substrate.

stripped setting the reader sees only the single cited step—no overview, no recent turns, no neighbors—on step-citing questions only.) A same-pipeline version closes that gap in scale: keeping the full deployed context and replacing only the router-selected verbatim appendix with LLM summaries of the same steps (same QA, overview, router, reader, and judge; the swap fires on 96% of questions) costs −5.1 pp (0.578 vs 0.629, same-day). Notably, the summarized appendix scores slightly below serving no appendix at all (gist-only 0.587): a lossy re-encoding of the selected evidence adds nothing over the one-line gists already in the overview. Verbatim, or nothing.

Versus standard vector-RAG. The embedding router is a standard vector-RAG baseline—it embeds each step's full text and takes cosine top-k, exactly the retrieval stage of Mem0/A-Mem/AMA-Agent—the only difference being that we re-inject the retrieved spans verbatim rather than a constructed store. On a shared subset its ordering is lexical 0.557 < vector-RAG 0.583 < RRF hybrid 0.600: our routing edges plain vector retrieval, but all three sit far above BM25 (0.3436) and every lossy-construction system (Table 2). The lesson is the same—the decisive gap is the verbatim-vs-lossy payload, not the retriever, so "standard RAG with verbatim re-injection" is already most of the win, and kvmemory adds tiering, structural pinning, and KV reuse on top.

Reader-side machinery saturates once routing is strong. If a one-sentence reader instruction is worth +5.5 pp, is the single-pass reader leaving more on the table? We stress-tested this with six reader-side mechanisms over a fixed flagship memory (same-batch controls; Table 8): letting the model compress the selected evidence before answering; a sufficiency gate fused into the answer prompt that can trigger re-retrieval; the same gate as a separate call; a query-conditioned entity-state timeline prepended as a scaffold; and deterministic lookup tools (exact get-by-index, exhaustive search) over the lossless store. None helps; several hurt. The one positive is re-retrieval under a weak (lexical) router (+1.3 pp), and the attribution is exact: the fused gate's −1.6 pp decomposes into −2.6 pp from the escape clause perturbing the answer prompt and +0.4 pp from the re-retrieval action itself. The pattern is uniform: a reader-side mechanism pays only when it injects information the selected context is actually missing. Once routing reaches $\sim 100\%$ evidence recall, the memory side is saturated—residual errors are backbone reading and reasoning failures, which none of this machinery (including verbatim tool access) repairs.

Two further probes sharpen the law. First, pre-computed reasoning does not transfer. If a one-sentence instruction that elicits decomposition is worth +5 pp, can the memory service cache that reasoning instead—produce an evidence plan (sub-questions, cited values, local conclusions) and hand it to a plain reader? No: prepending such a plan costs −5.6 pp under the default reader and −2.7 pp under the structured one (same-batch)—worse than no scaffold at all, and the more reasoning a scaffold carries, the more it hurts (plan −5.6 vs

Table 8: Reader-side mechanisms over a fixed flagship memory (AMA-Bench, same-batch Δ vs the plain single-pass reader; the plan row reports both readers: default / structured). Only re-retrieval under a weak router helps—i.e. only when the mechanism supplies specific, query-addressable evidence the selected context is missing; pre-computed reasoning and breadth-repair both fail.

Reader-side mechanism (memory fixed)	Δ accuracy
Model-selected evidence compression	−3.0 pp
Entity-state timeline scaffold	−1.6 pp
Sufficiency gate fused into the answer prompt (escape clause alone −2.6 pp; re-retrieval action alone +0.4 pp)	−1.6 pp
Sufficiency gate as a separate call	±0.0 pp
Deterministic lookup tools over the lossless store	+0.7 pp
Pre-computed reasoning plan (memory-side artifact)	−5.6 / −2.7 pp
Re-retrieval under budget starvation (6k cap, gate fires 19%)	−0.4 pp
Re-retrieval under a weak (lexical) router	+1.3 pp

state timeline −1.6; compression's −3.0 is a different failure—it removes context outright). The reader inherits the plan's errors and skips its own derivation: the instruction's value is eliciting the reader's own reasoning in the answering pass, not the reasoning content—which is why it cannot be moved into the memory. Second, re-retrieval cannot repair breadth starvation: at a 6k budget (a known −9.5 pp gap), a separate sufficiency gate fires on 19% of questions yet recovers nothing (−0.4 pp)—the budget cut removes the overview, and fetching more individual steps does not restore breadth. The law in final form: a reader-side mechanism pays only when the missing information is specific and query-addressable; reasoning must happen in the answering pass.

How much selected context: fill the budget. Within the routed context, more is monotonically better up to the window-forced cap: sweeping the assembled-context budget at fixed routing and reader gives 0.550 (6k tokens) → 0.624 (14k) → 0.644 (22k) on a 1,248-QA half (same batch; same-day replicate noise 0.5 pp). "Context rot" does not bite selected context at these scales—it bites unselected history (the 0.52 no-memory row of Table 2, and OpenWorld, where recency-truncated full context scores 0.474 vs routed 0.629)—so the rule is: select first, then fill the window with what was selected. The recency+gist overview earns its tokens (cutting to 6k costs −27 pp on Embodied and −23 pp on Web); the one exception is Text2SQL, which prefers compact context (+1–3 pp at smaller budgets), pointing to budget-adaptive routing as an open lever. The converse ablation removes query-time selection instead: answering from the recency +gist overview alone (no verbatim appendix) costs −4.3 pp (0.5865 vs 0.6290, same-day)—notably still above every constructed-memory system, but the routed verbatim appendix clearly earns its keep. Both halves of the assembled context are load-bearing.

4.5 Efficiency: KV sub-selection decouples cost from evidence

Serving the selected spans from the cache instead of re-encoding them decouples per-query cost from how much evidence a query selects. We position this section as a prototype microbenchmark—single-request, HF eager/SDPA kernels, no production serving-stack integration—establishing the mechanism, not a deployment result. We prefill a trajectory once and, per query, compare re-prefilling the header plus the selected spans (the text path) against gathering their cached KV and decoding only the question (sub-selection). Two facts hold across a dense Qwen3-8B and a 0.6B reader. First, sub-selection is faithful to the oracle it computes and empirically accuracy-neutral. The gathered KV is exactly the frozen model's key/value for the selected turns at their original positions, so sub-selection reproduces a full-prefill-then-mask-dropped oracle—not a re-prefill of the selected text alone, whose kept tokens would never have attended the dropped context (we measure that difference as a ∼13.5 logit gap to a compact re-prefill, ∼20× the residual below). Against the mask oracle the first decoded token matches on every query—100% first-token argmax on synthetic and on 144 real AMA-Bench QA (24/24 episodes)—at a ∼0.7 residual logit gap that is the usual bf16 split-vs-joint-prefill non-determinism. The agreement is sequence-level

Table 9: KV sub-selection decouples the per-query prefill from the selected-evidence size: the sub-selected decode re-prefills only the question (flat TTFT), while the text path re-prefills the evidence every query. Dense Qwen3-8B; first-token argmax matches the full-prefill oracle at every row, and end-to-end answers match the text path on this controlled synthetic set.

Selected / context (tok)	re-prefill tok/q	sub-select tok/q	TTFT (re-prefill → sub-select)	speedup
0.6k / 3.2k	575	20	66 ms → 41 ms	1.6×
1.9k / 4.3k	1916	20	189 ms → 40 ms	4.7×
3.0k / 8.1k	3008	20	295 ms → 41 ms	7.2×
4.4k / 13.5k	4387	20	448 ms → 41 ms	10.9×

and distributional, not just first-token: greedy-decoding 256 tokens, sub-selection and the mask oracle emit the identical full sequence on every query at both a 3.2k- and a 27k-token trajectory, at first-position logit-KL ≤ 0.011 to the oracle (median 0)—versus ~ 0.28 to a compact re-prefill, again showing the two are different computations—and the oracle's top-1-minus-top-2 answer margin averages 7.7 (min 0.25), so these are not knife-edge decisions. This full-sequence match survives the settings a reviewer would stress: it holds at 256-token generations (not just short answers), under an eager attention kernel as well as SDPA (6/6), and under a 4× YaRN RoPE extension (first-position logit-KL to the oracle ≤ 0.008, argmax 5–6/6)—as expected, since position-preserving gather is exact for RoPE regardless of the base frequency. A batched (ragged) gather we leave to future work. End-to-end, KV sub-selection and the deployed text pipeline are tied within noise on real AMA under a consistent reader/judge (-0.7 pp, one QA of 144)—as expected, since position-preserving reuse is a different computation from a compact re-prefill (the 86% first-token agreement between the two paths on that real-data run reflects that difference, not a defect). Second, because the sub-selected decode re-processes only the question, its time-to-first-token is flat while the text path grows with the selected evidence (Fig. 4, Table 9): the speedup rises from 1.6× to 10.9× as per-query evidence grows from ∼0.6k to ∼4.4k tokens, break-even is ∼1.5 queries per episode, and the one-time trajectory prefill saves ∼1.98M prefill tokens over a full open-ended AMA run. This flat TTFT is a prefill saving—the decode tokens still attend over the selected KV, so the win is avoiding re-encoding of the evidence, not eliminating attention over it. Pushing the selected-KV 7× beyond Table 9 confirms both sides: at 4k/16k/32k selected tokens the sub-selected TTFT stays flat ($\sim$ 80–140 ms) while the text path grows $0.4 \rightarrow 4.5$ s (speedup $4\times \rightarrow 32\times$), decode throughput stays ∼flat ($\sim$ 28 tok/s), and—as we flag—peak memory grows with the resident KV ($30 \rightarrow 37$ GB on an 80 GB A100), at a KV storage cost of 0.144 MB/token (9.4 GB for a 64k-token episode). A like-for-like comparison against production paged-attention/serving stacks needs a serving-system integration and is left to future work. The reuse itself applies to a frozen/processed corpus queried many times and to frozen early history reused across later turns; a continuously growing trajectory benefits from the same mechanism incrementally.

What a gathered KV block is (and is not). A selected KV block is a richer object than a copied text span: its keys/values were produced when the frozen reader processed the span in situ, inside the original trajectory, so gathered evidence carries the model's trajectory-conditioned interpretation of those tokens—context that a re-prefill of the span in isolation would not reconstruct (the ~ 13.5 logit gap above is that difference, measured). We view this contextual carryover as a property of activation-level memory, and we bound the claim in both directions. It is a statement about fidelity to the full-prefill computation, not a measured accuracy advantage (the two paths are tied within noise end-to-end; our headline benchmark numbers come from the deployed text pipeline, with KV sub-selection validated as its serving-layer equivalent at n=144 plus the logit-level gates above). And it cuts against a naive privacy reading: because selected KV is contextualized by unselected history, dropping a cold turn removes its text from the reader's context but not necessarily its influence from downstream cached KV (§6). Serving cost likewise still includes selected-KV residency, gather/copy, and decode attention over the selected tokens—sub-selection removes the per-query re-encode, nothing else.

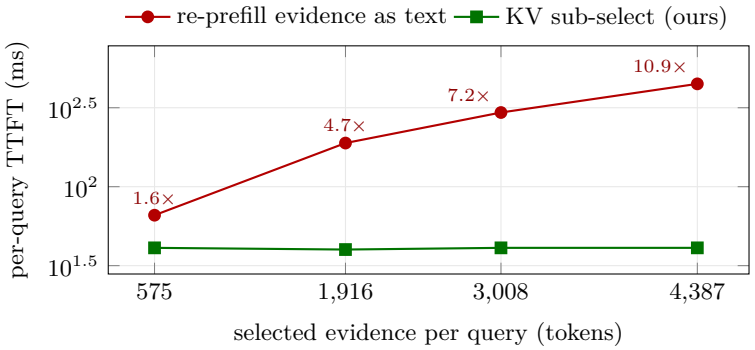

Figure 4: KV sub-selection decouples per-query latency from evidence size. As a query selects more evidence, re-prefilling it as text grows TTFT (66 → 448 ms; dense Qwen3-8B), whereas gathering the selected spans' KV re-prefills only the question and stays flat (∼40 ms); the widening gap is the per-query speedup (labeled, 1.6× → 10.9×), reaching 32× at 32k selected tokens (§4.5). First-token argmax matches the full-prefill oracle at every point.

Cross-backbone robustness (non-Qwen). Position-preserving KV reuse is a property of the gather, not the weights, so we expect it to hold on any RoPE model; we check one non-Qwen backbone rather than claim architecture-independence outright. Re-running the correctness/efficiency harness on Llama-3.1-8B-Instruct—a different architecture and vendor—reproduces the guarantees on our test set: GATE-A/B first-token argmax 6/6 across 48–200-step trajectories (residual $|\Delta\text{logit}| \leq 0.35$, in fact tighter than Qwen's ∼0.7), end-to-end match with text re-prefill on all 6 probes, and a flat ∼ 42 ms sub-select TTFT while the trajectory grows 3.1k→13k tokens (the one-time prefill grows 0.13 → 0.52 s but amortizes), i.e. 2.8× faster to first token at a fixed selection. These are small-n probes, not a broad multi-model study; the per-query prefill decoupling from trajectory length holds on both backbones we tested; the framework's accuracy contribution is separately evidenced across three benchmark families (§4.1, §4.2, §4.3) rather than a single backbone.

5 Related work

Agent-memory systems. The systems we compare to—AMA-Agent, Mem0 (Chhikara et al., 2025), MemoryBank (Zhong et al., 2024), A-Mem (Xu et al., 2025), MemGPT (Packer et al., 2023), Zep (Rasmussen et al., 2025), MemoRAG (Qian et al., 2024), HippoRAG (Gutiérrez et al., 2024)—share a trait: they construct memory at write time (extract facts, summarize, or build graphs) and retrieve from that lossy store. AMA-Bench's own needle ablation shows the cost: −41 pp from lossy construction. Two precedents are closest to us. Generative Agents (Park et al., 2023) retrieves by recency-importance-relevance over a stream that mixes verbatim observations with summarized reflections, but gives no verbatim-rehydration guarantee. MemGPT (Packer et al., 2023) pages verbatim context via LLM tool calls but recursively summarizes overflow rather than keeping it exactly rehydratable. kvmemory differs by keeping raw spans as the authoritative payload, routing to them, and serving gists only as a one-line index and overview scaffold—never as a substitute for evidence a query needs, which is always rehydrated verbatim. Crucially, all of these are query-aware, so query-awareness is not the novelty; the combination of verbatim payload + cheap routing + gist-as-index is.

File- and transcript-based memory. A recent line of practice keeps raw history as-is and lets an agent search it with tools: filesystem-backed agents over stored transcripts report strong LOCOMO numbers (Letta Team, 2025; Pam Research, 2025), and AMA-Agent's tool loop over the full trajectory JSON is the same idea on agent traces. These systems are consistent with our thesis—they, too, refuse to compress the payload—and we do not claim priority on "keep the raw text." Our contribution relative to them is the controlled decomposition (payload-fidelity causality, the memory×reader confound, structural addressing as a separate

channel) and the serving primitive (position-preserving KV sub-selection), which text-file replay does not provide.

Retrieval-augmented generation and agent frameworks. Classic RAG retrieves external passages and conditions the reader on them—RAG (Lewis et al., 2020), REALM (Guu et al., 2020), RETRO (Borgeaud et al., 2022), Atlas (Izacard et al., 2023). kvmemory is RAG turned inward: the corpus is the agent's own trajectory and the retrieved units are re-injected as verbatim KV rather than re-encoded text, so retrieval and reuse share one representation (the "standard vector-RAG" point of this design is our EmbeddingRouter, §4.4). The memory modules practitioners actually deploy—the buffer/summary/vector stores in LangChain (Chase, 2022) and AutoGen (Wu et al., 2023)—occupy exactly the slot kvmemory targets; it is a drop-in, verbatim-first alternative to those summary/vector defaults.

KV reuse and compression. A systems line reuses KV caches for latency—prefix caching in vLLM (Kwon et al., 2023) and SGLang (Zheng et al., 2024)—which kvmemory sits on top of. A compression line removes tokens via eviction (e.g. H2O (Zhang et al., 2023)) or learned summarization (RECOMP (Xu et al., 2024)); these optimize the cache or the prompt, not the selection-quality memory question that determines whether a multi-hop answer survives. A parallel architecture line compresses or retrieves past activations inside the model—Compressive Transformer (Rae et al., 2020), Memorizing Transformer (Wu et al., 2022)—whereas kvmemory sits one level up, as a training-free memory manager over the raw KV of a frozen model. Our point is orthogonal and, where they overlap (at a matched training-free, query-free budget), extractive retention is already at least as good as generative summary—so the lever is selection, not harder compression.

6 Limitations: when does verbatim selection win?

kvmemory defers answer-extraction to the reader at inference time. This is ideal for a capable, self-hosted reader and for dense/long-horizon evidence—where it wins (AMA-Bench, LOCOMO). It has two boundaries. First, scope: the strongest form of our efficiency claim—position-preserving KV sub-selection of arbitrary spans—needs self-hosting; hosted APIs expose only contiguous-prefix prompt-caching, which our prefix-friendly layout still benefits from but which cannot gather arbitrary spans. Second, reader capacity: for a weak reader over a benchmark whose answers are tiny isolated facts, systems that extract atomic facts at construction time can present an easier context—the gap is a matter of reader capacity, not retrieval (our router surfaces the evidence 94% of the time). This is also why we report LongMemEval-S only in the strong-reader (Qwen3-32B) regime: it is a scoping choice—testing the settings the claim is about, capable self-hosted readers—not a counterexample we hide.

Third, once routing is strong, the reader is the residual bottleneck—and a different answer-production paradigm owns one domain. On AMA-Bench Software, selection achieves 100% recall of the asked steps and deterministic tool access adds nothing (§4.1), yet AMA-Agent leads there by +17 pp through an iterative loop that retrieves, executes code over the full trajectory, verifies, and composes from tool outputs. The value is the loop, not its verbatim output style: a quote-first single-pass imitation of that style costs −10 pp (§4.1). Our reader-side ablations (§4.4) indicate this cannot be absorbed by bolting machinery onto a single-pass reader; it is a genuinely complementary design, not a retrieval gap.

Fourth, verbatim storage has governance costs. Unlike lossy construction, the store retains raw history: PII persists verbatim and must be handled by policy outside the memory (encryption at rest, retention windows, write-time injection scanning—verbatim text stored is verbatim text replayed to the reader). Deletion semantics differ by layer. In the text substrate, dropping a turn's segment is exact: the next assembly simply no longer contains it, with no derived summaries or graph edges to chase down. In the KV layer it is not: cached keys/values are contextualized, so the KV of tokens after an erased turn was computed while attending to it, and exact erasure requires invalidating or recomputing the downstream blocks (bounded, in our per-episode caches, by the episode length). We make no privacy claim for cached KV, and we have not evaluated redaction pipelines.

7 Conclusion

For long-horizon agents whose answers are exact, objective, and buried in dense trajectories—and whose histories overflow the window or drown the reader in distractors—the right memory operation is to select, not construct: keep the raw history as reusable KV and rehydrate verbatim the spans a query needs. When the history fits and the reader can exploit it, full context can win; when the reader is weak and the facts are tiny, construction can win; we state both boundaries rather than universalize the slogan. Within its regime, a small framework built on this principle leads our runs of AMA-Bench (official verification pending), is competitive with dedicated dialogue-memory systems on LOCOMO, and generalizes to LongMemEval-S—with the win driven by payload fidelity rather than router sophistication, reader-side effects measured and reported separately, and the cost amortized by KV reuse. The recipe is simple enough to fork, and we offer it, with configurations and predictions released, as a reproducible baseline that centers payload fidelity over memory construction.

References

AMA-Bench Team. AMA-Bench: Benchmarking agent memory over arbitrary-length trajectories. In International Conference on Learning Representations (ICLR), 2026. arXiv:2602.22769.

Sebastian Borgeaud et al. Improving language models by retrieving from trillions of tokens. In International Conference on Machine Learning (ICML), 2022. arXiv:2112.04426.

Harrison Chase. LangChain: Building applications with LLMs through composability, 2022. Software, github.com/langchain-ai/langchain.

Prateek Chhikara et al. Mem0: Building production-ready ai agents with scalable long-term memory. European Conference on Artificial Intelligence (ECAI), 2025. arXiv:2504.19413.

Bernal Jiménez Gutiérrez et al. HippoRAG: Neurobiologically inspired long-term memory for large language models. In Advances in Neural Information Processing Systems (NeurIPS), 2024. arXiv:2405.14831.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-augmented language model pre-training. In International Conference on Machine Learning (ICML), 2020. arXiv:2002.08909.

Gautier Izacard et al. Atlas: Few-shot learning with retrieval augmented language models. Journal of Machine Learning Research (JMLR), 2023. arXiv:2208.03299.

Woosuk Kwon et al. Efficient memory management for large language model serving with PagedAttention. In ACM Symposium on Operating Systems Principles (SOSP), 2023.

Letta Team. Benchmarking AI agent memory: Is a filesystem all you need? https://www.letta.com/blog/benchmarking-ai-agent-memory/, 2025. Filesystem-backed agent memory over raw transcripts; reports LOCOMO 0.740 with gpt-4o-mini.

Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In Advances in Neural Information Processing Systems (NeurIPS), 2020. arXiv:2005.11401.

Adyasha Maharana et al. Evaluating very long-term conversational memory of LLM agents. In Annual Meeting of the Association for Computational Linguistics (ACL), 2024. arXiv:2402.17753.

Charles Packer et al. MemGPT: Towards llms as operating systems. arXiv preprint arXiv:2310.08560, 2023.

Pam Research. Pam outperforms ChatGPT by 40% on LoCoMo benchmark. https://manager.harmix.ai/blog/pam-locomo-benchmark-results, 2025. File-first memory; reports LOCOMO 0.744.

Joon Sung Park et al. Generative agents: Interactive simulacra of human behavior. In ACM Symposium on User Interface Software and Technology (UIST), 2023. arXiv:2304.03442.

Hongjin Qian et al. MemoRAG: Moving towards next-gen rag via memory-inspired knowledge discovery. arXiv preprint arXiv:2409.05591, 2024.

Qwen Team. Qwen3 technical report. In arXiv preprint, 2025. arXiv:2505.09388.

Jack W. Rae et al. Compressive transformers for long-range sequence modelling. In International Conference on Learning Representations (ICLR), 2020. arXiv:1911.05507.

Preston Rasmussen et al. Zep: A temporal knowledge graph architecture for agent memory. arXiv preprint arXiv:2501.13956, 2025.

Di Wu et al. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In International Conference on Learning Representations (ICLR), 2025. arXiv:2410.10813.

Qingyun Wu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155, 2023.

Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In International Conference on Learning Representations (ICLR), 2022. arXiv:2203.08913.

Fangyuan Xu, Weijia Shi, and Eunsol Choi. RECOMP: Improving retrieval-augmented lms with compression and selective augmentation. In International Conference on Learning Representations (ICLR), 2024. arXiv:2310.04408.

Wujiang Xu et al. A-Mem: Agentic memory for llm agents. In Advances in Neural Information Processing Systems (NeurIPS), 2025. arXiv:2502.12110.

Zhenyu Zhang et al. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In Advances in Neural Information Processing Systems (NeurIPS), 2023. arXiv:2306.14048.

Lianmin Zheng et al. SGLang: Efficient execution of structured language model programs. Advances in Neural Information Processing Systems (NeurIPS), 2024. arXiv:2312.07104.

Wanjun Zhong et al. MemoryBank: Enhancing large language models with long-term memory. In AAAI Conference on Artificial Intelligence, 2024. arXiv:2305.10250.